JavaScript

ALEX CHANG & DANIEL COBLENTZ

12.4.2025

PROGRAMMING LANGUAGES: CS 471

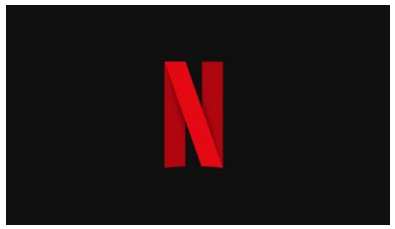
Content Outline:

1) Introduction to JavaScript
2) Applications
3) Distinct language feature: Hoisting
4) Evaluation

Introduction

- **Who and when:** Brendan Eich invented JavaScript in 1995 for Netscape 2, a discontinued web browser
  - In 1997, it became an ECMA standard (ECMA-262)
    - Became the basis for ECMAScript: a standardized scripting language, of which JavaScript is a dialect
- **Why:** To be a simple language; adds interactivity and dynamic behavior to static web pages and can run directly in web browsers
- **Most recent version:** ECMAScript 2025 (ES16) - released June 2025
  - As of 2018, JavaScript is supported by every major browser

Application Domains



1) Frontend web development
   - Netflix, Airbnb all build with React, JavaScript framework
   - Powers dynamic content, animations, user interactions on millions of sites
   - Frameworks: React, Angular, Vue, Svelte, Ember, Backbone
2) Backend and server-side development
   - PayPal: handles over 450 million users
   - Uber: responsible for processing millions of ride requests and real time location tracking and sharing.
   - Frameworks: Express, Nest, Adonis, Node

Distinct feature: Hoisting

What is hoisting?

- Hoisting is JavaScript's default behavior of moving declarations to the top of their scope during compilation
- Variables and functions can be used before they are declared in the code
- Only declarations are hoisted, not the initialization

How it works

Before executing the code, JS scans for all variable and function declarations. These declarations are moved to the top of their scope (global or function level). Then the code runs as if declarations were written at the beginning.

Simple hoisting example

```
sayHello();

    function sayHello() {
      console.log("Hello!");
}
```

Function call

Function declaration

* Functions are fully hoisted, so you can call them before declaration.

Practical use for hoisting (code organization)

```
function processUserOrder(orderID) {
    const order = fetchOrder(orderID);
    const validated = validateOrder(order);
    const total = calculateTotal(validated);
 return submitPayment(total);
}
function fetchOrder(ID) { /* ... */ }
function validateOrder(order) { /* ... */ }
function calculateTotal(order) { /* ... */ }
function submitPayment(amount) { /* ... */ }
```

Main logic, executed after hoisted functions

Helper functions, placed at the bottom, but hoisting executes them first

Language evaluation

- **Simplicity/orthogonality:** C-style syntax, easy to learn and adapt
- **Control structures:** Conditionals, loops, imperative statements
- **Data types:** Primitives (string, number, boolean, null, undefined, symbol, BigInt), references (objects, arrays, functions)
- **Syntax design:** Clean and readable; arrow functions, template literals, destructuring
- **Abstraction support:** Functions, supports OOP and functional programming
- **Expressivity:** Concise syntax for complex operations
- **Type checking:** Weakly typed language, runtime errors instead of compile time catches, flexible for rapid development

Language Evaluation, cont.

Strengths:

1) Universal compatibility.
2) Large community support and yearly updates.
3) Beginner-friendly syntax.
4) Wide variety of libraries and frameworks.

Weaknesses:

1) Hoisting confusion: Unique behaviors like hoisting create unexpected bugs and require deep understanding of the feature.
2) Legacy baggage: Backward compatibility means some outdated features such as =, ==. === persists alongside modern ones.
3) Performance limitations: Single-threaded execution can bottleneck CPU-intensive operations.
4) Weak typing: Type coercion leads to runtime errors developers usually switch to TypeScript for large projects.

Links

- https://developer.mozilla.org/en-US/docs/Web/JavaScript https://www.w3schools.com/js/js_history.asp
- https://www.geeksforgeeks.org/javascript/introduction-to-javascript/
- https://www.interviewbit.com/blog/javascript-features/
- https://dev.to/itsshaikhaj/deep-dive-into-javascript-closures-how-and-when-to-use-them-5c63
- https://dev.to/rohith_nag/inside-nodejs-a-deep-dive-into-v8-libuv-the-event-loop-thread-pool-5fcn
- https://stackoverflow.com/questions/912479/what-is-the-difference-between-javascript-and-ecmascript
- https://www.freecodecamp.org/news/whats-the-difference-between-javascript-and-ecmascript-cba48c73a2b5/
- https://262.ecma-international.org/16.0/index.html?_gl=1*dgo64i*_ga*MTQwMTg4MDg5MC4xNzY0NTMyMjYw*_ga_TDCK4DWEPP*czE3NjQ1MzIyNjAkbzEkZzAkdDE3NjQ1MzIyNjAkajYwJGwwJGgw
- ECMAScript: https://tc39.es/ecma262/#sec-normative-references
- https://www.geeksforgeeks.org/javascript/history-of-javascript/
- jQuery history: https://jquery.org/history/
- JS history: https://www.w3schools.com/js/js_history.asp#:~:text=JavaScript%20was%20invented%20by%20Brendan,JavaScript%20for%20the%20Firefox%20browser.
- Control structures: https://metana.io/blog/javascript-control-structures/
- JavaScript history: https://deno.com/blog/history-of-javascript